

# Week-4\_separated

June 11, 2026

## 1 Import the packages

```
[1]: import pandas as pd
import numpy as np
from scipy.integrate import quad
from scipy.optimize import minimize
```

## 2 Load and Split the Dataset

```
[2]: # Load the dataset
file_path = 'data/Pantheon+SHOES.dat'
try:
    df = pd.read_csv(file_path, sep=r'\s+')

    # Assign the diagonal error column
    if 'm_b_corr_err_DIAG' in df.columns:
        df['error_m_b'] = df['m_b_corr_err_DIAG']
    elif 'mBERR' in df.columns:
        df['error_m_b'] = df['mBERR']
    else:
        df['error_m_b'] = 0.1

    # Split the data into the two distinct subsets
    df_calib = df[(df['IS_CALIBRATOR'] == 1) & (df['CEPH_DIST'] > -9)].copy()
    df_cosmo = df[df['IS_CALIBRATOR'] == 0].copy()

    print(f"Dataset loaded successfully.")
    print(f" - Calibrator Subsample Size: {len(df_calib)}")
    print(f" - Cosmological Subsample Size: {len(df_cosmo)}")

except FileNotFoundError:
    print(f"Error: The file {file_path} was not found.")
```

Dataset loaded successfully.

- Calibrator Subsample Size: 77
- Cosmological Subsample Size: 1624

### 3 Define Physics Model

```
[3]: c = 299792.458 # Speed of light in km/s

def model_calibrators(M_B, mu_ceph):
    return M_B + mu_ceph

def E_inverse(z, Omega_m):
    Omega_L = 1.0 - Omega_m
    return 1.0 / np.sqrt(Omega_m * (1 + z)**3 + Omega_L)

def luminosity_distance_mpc(z_hel, z_cmb, H_0, Omega_m):
    integral, _ = quad(E_inverse, 0, z_cmb, args=(Omega_m,))
    return (c / H_0) * (1 + z_hel) * integral

def model_cosmology(z_hel, z_cmb, M_B, H_0, Omega_m):
    d_L_mpc = luminosity_distance_mpc(z_hel, z_cmb, H_0, Omega_m)
    mu_lcdm = 5.0 * np.log10(d_L_mpc) + 25.0
    return M_B + mu_lcdm
```

### 4 Analysis 1 - Cepheid Calibrators Only

```
[4]: def chi_square_calib(M_B_array, data):
    M_B = M_B_array[0] # scipy minimize expects an array
    chi2 = 0.0
    for _, row in data.iterrows():
        m_b_obs = row['m_b_corr']
        sigma = row['error_m_b']
        mu_ceph = row['CEPH_DIST']

        m_b_th = model_calibrators(M_B, mu_ceph)
        chi2 += ((m_b_obs - m_b_th) ** 2) / (sigma ** 2)
    return chi2

print("Running fit on Cepheid Calibrators...")
res_calib = minimize(chi_square_calib, x0=[-19.0], args=(df_calib,),
                     method='L-BFGS-B', bounds=[(-21.0, -18.0)])

if res_calib.success:
    best_M_B = res_calib.x[0]
    print("\n--- Calibrator Fit Results ---")
    print(f"Absolute Magnitude (M_B) = {best_M_B:.4f}")
    print(f"Minimum Chi-Square = {res_calib.fun:.2f}")
else:
    print("Calibrator fit failed.")
```

Running fit on Cepheid Calibrators...

```

--- Calibrator Fit Results ---
Absolute Magnitude (M_B) = -19.2517
Minimum Chi-Square = 12.70

```

## 5 Analysis 2 - Cosmological Supernovae Only

```

[5]: def chi_square_cosmo(params, data):
    Omega_m, H_0, M_B = params
    if Omega_m <= 0 or Omega_m >= 1 or H_0 <= 0:
        return np.inf

    chi2 = 0.0
    for _, row in data.iterrows():
        m_b_th = model_cosmology(row['zHEL'], row['zCMB'], M_B, H_0, Omega_m)
        chi2 += ((row['m_b_corr'] - m_b_th) ** 2) / (row['error_m_b'] ** 2)
    return chi2

print("Running fit on Cosmological Supernovae (Non-Calibrated)...")
# Initial guesses: [Omega_m, H_0, M_B]
initial_guess = [0.5, 80.0, -19.5]
bounds = ((0.01, 0.99), (50.0, 100.0), (-21.0, -18.0))

res_cosmo = minimize(chi_square_cosmo, x0=initial_guess, args=(df_cosmo,),
    method='L-BFGS-B', bounds=bounds)

if res_cosmo.success:
    print("\n--- Cosmological SNe Fit Results ---")
    print(f"Matter Density (Omega_m) = {res_cosmo.x[0]:.4f}")
    print(f"Hubble Constant (H_0) = {res_cosmo.x[1]:.2f} km/s/Mpc")
    print(f"Absolute Magnitude (M_B) = {res_cosmo.x[2]:.4f}")
    print(f"Minimum Chi-Square = {res_cosmo.fun:.2f}")
else:
    print("Cosmological fit failed.")

```

Running fit on Cosmological Supernovae (Non-Calibrated)...

```

--- Cosmological SNe Fit Results ---
Matter Density (Omega_m) = 0.3756
Hubble Constant (H_0) = 74.03 km/s/Mpc
Absolute Magnitude (M_B) = -19.2077
Minimum Chi-Square = 751.72

```